Students: Naji Bou Zeid 231833 & Mathieu Moussa 231284

IoT Intrusion Detection System : CICIoT2023

Machine Learning | Multi-Class Network Threat Classification

Instructor: Dr. Youssef Bakouny**Program:** Computer & Communications Engineering CCE Year 4**Date:** April 2026

1. DATA SOURCE

The project uses the **CICIoT2023** dataset, a large-scale benchmark for IoT network intrusion detection published by the Canadian Institute for Cybersecurity. The raw data consists of **63 CSV files** of captured network flow records generated from a realistic IoT testbed comprising 105 IoT devices under 33 distinct attack scenarios plus benign traffic.

Raw dataset: 47 million flow records, 39 raw features + 1 label column.
Working subset: Stratified 5% sample (1.3M rows) preserving class balance.
Final split: 70% train / 15% validation / 15% test.

Classes: 34 (1 Benign + 33 attack types including DDoS, DoS, Mirai botnet, Recon, Brute-Force, Web attacks, MITM, and Malware).
Features after selection: 39 network-flow statistics.

2. APPROACH : PROJECT PIPELINE

Step	Notebook	Description
Step 1	01 Ingestion	Loaded and merged all 63 raw CSV files after schema consistency validation. Inspected data types, identified and removed infinite values, NaN entries, and duplicate rows. Analysed class distribution and created a representative stratified subset for downstream use (using a 35K cap rule for frequent classes).
Step 2	02 Preprocessing	Stratified 70/15/15 train/val/test split. Applied four preprocessing variants — `raw` `out` `scaled` `out_scaled` to support different model families. Performed IQR-based outlier clipping (fit on train only), StandardScaler z-score normalisation, skewness correction, and multi-method feature selection (constant feature removal, Pearson correlation pruning, mutual information ranking). Computed inverse-frequency class weights to address severe class imbalance.
Step 3	03 Logistic Regression	Multinomial Logistic Regression with LBFGS solver and L2 regularisation. Regularisation strength C tuned via GridSearchCV on a fixed PredefinedSplit (no data leakage), scored with Macro-F1. Best: `C = 10.0` on `out_scaled` features. Final model retrained on train + validation before test evaluation.
Step 4	04 Random Forest	Ensemble of decision trees with bootstrap aggregation. Hyperparameters tuned via GridSearchCV: number of estimators, maximum depth, and features per split. Best: `n_estimators=200` `max_depth=40` `max_features=sqrt` on `scaled` features. Feature importance analysis identified Rate, IAT, and Tot sum as most discriminative.
Step 5	05 Deep Neural Network	PyTorch feedforward DNN — 3 hidden layers (256 → 128 → 64 neurons) with ReLU activations, Batch Normalization, and Dropout regularisation. Optimised with Adam and weighted CrossEntropyLoss. Early stopping (patience = 10 epochs) on validation loss. Best: `lr=0.001` `dropout=0.2` `95 epochs` on `out_scaled` features.
Step 6	06 Perceptron	Linear Perceptron with One-vs-Rest (OvR) multi-class strategy. Tuned regularisation penalty and strength via GridSearchCV. Best: `L2, α=0.0001` on `out_scaled` features. No probability output (hard predictions only); Log Loss not applicable.
Steps 7	Deployment Simulation / Drift and Alert Monitoring	Production serving layer built with **FastAPI** exposing REST endpoints for inference, model metrics, and confusion matrices. Real-time **Streamlit** dashboard IDS Command Center providing live classification, multi-model benchmarking, and a SOC Monitor with statistical drift detection PSI, chi-squared, KL divergence, confidence trend analysis.

3. SUMMARY OF METHODS

Preprocessing

IQR clipping on high-outlier features; StandardScaler; skewness correction; constant & near-constant feature removal; Pearson correlation pruning; Mutual Information ranking. All transformations fit exclusively on training data and applied identically to validation and test sets.

Evaluation Protocol

Primary metric: **Test Macro-F1** (equal weight per class regardless of frequency). Secondary: Accuracy, Macro-Precision, Macro-Recall, Log Loss, and Benign False-Positive Rate (FPR). Hyperparameter selection via PredefinedSplit GridSearchCV — test set never touched until final evaluation. All models retrained on train + val before test assessment.

4. RESULTS SUMMARY

Model	Preprocessing	Test Accuracy	Test Macro-F1	Test Precision	Test Recall	Benign FPR	Log Loss
Random Forest ★	`scaled`	**75.49%**	**0.6437**	**0.6850**	**0.6356**	**29.3%**	**0.6437**
Deep Neural Network	`out_scaled`	70.90%	0.6027	0.6324	0.6447	41.6%	0.7110
Logistic Regression	`out_scaled`	66.33%	0.5498	0.5708	0.5841	53.2%	0.9309
Perceptron	`out_scaled`	54.21%	0.4159	0.4887	0.4619	60.9%	N/A

★ **Random Forest** achieved the best overall performance with a Test Macro-F1 of **0.6437** and the lowest Benign FPR of **29.3%**, making it the most deployment-ready model. The DNN achieved the highest recall (0.6447) but at the cost of a higher false-positive rate. All models faced difficulty on rare classes such as Recon-PingSweep, XSS, Uploading Attack, and SQL Injection due to limited training examples, a challenge partially mitigated by balanced class weighting. The 34-class problem makes these results somewhat consistent with the CICIoT2023 paper's reported baselines.